

Problem 1

A)

1. Once the hashed passwords is exposed, the adversary can use a brute-force attack to figure out the origin passwords.
2. As for Merkle-Damgard construction, it is vulnerable to length extension attack especially when key is exposed.

B)

If all users share the same hash function, then the adversary can use each brute-force attempt to guess all the passwords on one time.

C)

Typically, iterating a hash function is regarded as a good way to slow down brute-force attacks.

D)

I'll set the iteration count to be a relative high number, e.g., 10 million

E)

I'll set the iteration count to be a relative small number, e.g., 1000 Because Always, Our computer is weak in check the submitted pw compared to the server. Second, the throughput of the "popular server" is much higher than PC which at the same time results in exposing to a high level of being attacked.

Problem 2

B)

If a user chooses a uniformly random string of 20 letters (a-z) as their password, and we hash the password with a standard cryptographic hash function with 256-bit output, how many guesses on average will it take to recover their password

There exists a total of 26^{20} possible composition of the passwords, considering the definition of "a standard cryptographic hash function", we use an brute-force attack to guess the pw, it should take $26^{20}/2$ guesses to find the correct one on average.

D)

The cost will be extremely enhanced. Each guess will be a feasible try only if it "coincidentally" matches the right salt of the corresponding pw in the salt space S .

Problem 4

A)

we can randomly select one pw, and then iterate the function H on it. For each new-generated digest, we matches it with the digest generated before. Always, the iterate falls into a cycle ,assumed that the cycle is of l -length, for the first digest value to be duplicated, denoted as x_k , then it comes into that $x_{k-1}! = x_{k-1+l}$ but $H(x_{k-1}) = H(x_{k-1+l})$

complements: use Floyd's Cycle Detection Algorithm to find the specific point x_k